OS Lab Project

fork() & pipe() — Code Documentation

Sequential vs Parallel Compute

Files covered:

`compute.h`	shared header — types and function declarations
`parse.h`	file parser — reads numbers from disk
`sequential_compute.c`	sequential fold over all numbers
`parallel_compute.c`	parallel version using fork and pipes
`test_correctness.c`	test suite that verifies everything is correct

Deadline: Monday, March 23 — Grade: 14%

Contents

1 Quick concept recap before reading the code **2**
1.1 What a pipe is . . . 2
1.2 What fork does . . . 2

2 compute.h — shared header **3**

3 parse.h — file parser **4**

4 sequential_compute.c **8**

5 parallel_compute.c **10**
5.1 Step 1 — read the numbers . . . 10
5.2 Step 2 — create the pipes . . . 10
5.3 Step 3 — compute slice sizes . . . 11
5.4 Step 4 — fork the children . . . 12
5.5 Step 4a — inside the child . . . 12
5.6 Step 5 — parent sends slice boundaries . . . 13
5.7 Step 6 — wait for children, then collect results . . . 14
5.8 Step 7 — aggregate the partial results . . . 15
5.9 Step 8 — final cleanup . . . 15

6 test_correctness.c **16**
6.1 write_file — helper to create test input files . . . 16
6.2 check — test assertion helper . . . 16
6.3 test_sequential . . . 16
6.4 test_parallel . . . 17
6.5 test_both_agree . . . 17
6.6 main . . . 18

7 Summary of the full data flow **19**

8 Common mistakes this code avoids **19**

1 Quick concept recap before reading the code

1.1 What a pipe is

A pipe is a one-directional channel in RAM that two processes can use to talk to each other. You create it with `pipe(fd)`, which gives you two file descriptors:

Descriptor	Name	What you do with it
`fd[0]`	read end	call `read()` on this to receive data
`fd[1]`	write end	call `write()` on this to send data

Note

A pipe is like a one-way tube. Data goes in one end and comes out the other. You cannot read from `fd[1]` or write to `fd[0]` — the direction is fixed.

Important

Always close the end you are **not** using. If you forget, `read()` will block forever because the OS thinks someone might still write to the pipe later.

1.2 What fork does

`fork()` creates an exact copy of the current process. After the call there are two processes running the same program. The only difference is the return value:

- **Returns 0** — you are inside the child process.
- **Returns a positive number** — you are the parent, and the number is the child's PID.
- **Returns -1** — something went wrong, no child was created.

This is used in `parallel_compute` to spin up multiple worker processes.

2 compute.h — shared header

```
1  #ifndef COMPUTE_H
2  #define COMPUTE_H
3
4  typedef int (*func_t)(int a, int b);
5
6  int sequential_compute(const char *filepath, func_t f);
7
8  int parallel_compute(const char *filepath, int n_proc, func_t f
       );
9
10 #endif
```

Listing 1: compute.h

Line by line

Lines 1–2 — include guard

`#ifndef COMPUTE_H` and `#define COMPUTE_H` form what is called an *include guard*. If this header gets included more than once in the same compilation unit, the second inclusion is skipped completely. Without this you would get duplicate definition errors. Line 8 closes the guard with `#endif`.

Line 4 — the function pointer type

```
typedef int (*func_t)(int a, int b);
```

This creates a new type called `func_t`. It means: "a pointer to any function that takes two `int` arguments and returns an `int`." Instead of writing the full pointer syntax everywhere, you just write `func_t f`.

Line 6 — declaration of sequential_compute

```
int sequential_compute(const char *filepath, func_t f);
```

This tells the compiler that `sequential_compute` exists somewhere and takes two parameters: a path to a file, and a function pointer. The actual implementation is in `sequential_compute.c`.

Line 8 — declaration of parallel_compute

Same idea but with an extra `int n_proc` parameter — how many child processes to use.

3 parse.h — file parser

This file contains one function, `read_numbers`, that reads all integers from a text file and returns them as a dynamically allocated array. It handles both newline-separated and comma-separated files.

```c
#ifndef PARSE_H
#define PARSE_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

static int *read_numbers(const char *filepath, int *count) {
    FILE *fp = fopen(filepath, "r");
    if (!fp) {
        perror("fopen");
        return NULL;
    }

    int capacity = 64;
    int *nums = malloc(capacity * sizeof(int));
    if (!nums) { fclose(fp); return NULL; }

    *count = 0;
    char buf[256];

    while (fgets(buf, sizeof(buf), fp)) {
        char *tok = strtok(buf, ",\n\r ");
        while (tok) {
            int valid = 0;
            for (int i = 0; tok[i]; i++)
                if (isdigit(tok[i]) || tok[i] == '-') { valid = 1; break; }

            if (valid) {
                if (*count == capacity) {
                    capacity *= 2;
                    int *tmp = realloc(nums, capacity * sizeof(int));
                    if (!tmp) { free(nums); fclose(fp); return NULL; }
                    nums = tmp;
                }
                nums[(*count)++] = atoi(tok);
            }
            tok = strtok(NULL, ",\n\r ");
        }
    }

    fclose(fp);
```

```
44     return nums;
45 }
46
47 #endif
```

Listing 2: parse.h — full file

Line by line

Lines 4–7 — includes

- `stdio.h` — needed for `fopen`, `fgets`, `perror`.
- `stdlib.h` — needed for `malloc`, `realloc`, `free`, `atoi`.
- `string.h` — needed for `strtok`.
- `ctype.h` — needed for `isdigit`.

Line 9 — function signature

```
1 static int *read_numbers(const char *filepath, int *count)
```

`static` means this function is only visible inside the file that includes this header — it is not exported to other translation units. `int *count` is a pointer to an integer; the function will write the number of elements it found into whatever `count` points to.

Line 10 — opening the file

```
1 FILE *fp = fopen(filepath, "r");
```

`fopen` opens the file at `filepath` for reading. It returns a pointer to a `FILE` structure, or `NULL` if the file does not exist or cannot be opened.

Lines 11–13 — error check

If `fp` is `NULL`, `perror` prints a human-readable error message to `stderr`, and we return `NULL` to signal failure to the caller.

Lines 16–17 — dynamic array setup

```
1 int capacity = 64;
2 int *nums = malloc(capacity * sizeof(int));
```

We start with space for 64 integers. We do not know how many numbers are in the file yet, so we allocate a starting buffer and grow it as needed. `malloc` reserves memory on the heap and returns a pointer to it.

Line 18 — null check on malloc

If `malloc` fails (out of memory), it returns `NULL`. We close the file and return `NULL`.

Line 20 — initialise count to zero

`*count = 0` writes zero into the integer that the caller passed in. We will increment this as we read numbers.

Line 23 — read lines one at a time

```
while (fgets(buf, sizeof(buf), fp))
```

`fgets` reads one line from the file into `buf`, including the newline character. It returns `NULL` when there is nothing left to read, which ends the loop.

Line 24 — split the line into tokens

```
char *tok = strtok(buf, ",\n\r ");
```

`strtok` splits `buf` into tokens using the characters `,`, newline, carriage return, and space as separators. This is how we handle both comma-separated and newline-separated files with a single call. It returns a pointer to the first token, or `NULL` if there are none.

Lines 26–28 — validate the token

```
int valid = 0;
for (int i = 0; tok[i]; i++)
    if (isdigit(tok[i]) || tok[i] == '-') { valid = 1; break; }
```

We scan each character of the token. If we find at least one digit or a minus sign, the token is a valid number. This skips blank tokens or any stray whitespace that `strtok` may have produced.

Lines 30–35 — grow the array if needed

```
if (*count == capacity) {
    capacity *= 2;
    int *tmp = realloc(nums, capacity * sizeof(int));
    if (!tmp) { free(nums); fclose(fp); return NULL; }
    nums = tmp;
}
```

If we have filled the array, we double the capacity and call `realloc` to resize the block of memory. We use a temporary pointer `tmp` because if `realloc` fails it returns `NULL` — if we had written `nums = realloc(nums, ...)`, we would have lost the original pointer and caused a memory leak.

Line 36 — store the number

```
nums[(*count)++] = atoi(tok);
```

`atoi` converts the string token to an integer. We store it in the array at index `*count`, then increment `*count`. The `*count++` happens after the assignment due to operator precedence.

Line 38 — get the next token

```
tok = strtok(NULL, ",\n\r ");
```

Calling `strtok` with `NULL` as the first argument tells it to continue from where it left off in the same string. This is how you iterate through all tokens.

Lines 42–43 — close file and return

`fclose(fp)` releases the file handle. We return `nums`, the pointer to the array of integers. The caller is responsible for calling `free(nums)` when done.

4 sequential_compute.c

This is the simpler of the two compute functions. No forking, no pipes. Just read the numbers and apply the function one step at a time.

```
1  #include "compute.h"
2  #include "parse.h"
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  int sequential_compute(const char *filepath, func_t f) {
7      int count = 0;
8      int *nums = read_numbers(filepath, &count);
9      if (!nums || count == 0) {
10         free(nums);
11         return 0;
12     }
13
14     int result = nums[0];
15     for (int i = 1; i < count; i++) {
16         result = f(result, nums[i]);
17     }
18
19     free(nums);
20     return result;
21 }
```

Listing 3: sequential_compute.c — full file

Line by line

Lines 1–4 — includes

`"compute.h"` gives us the `func_t` type and the function declaration. `"parse.h"` gives us `read_numbers`. The standard headers are for `free`.

Line 6 — function signature

```
1  int sequential_compute(const char *filepath, func_t f)
```

Returns an `int` — the final result of applying `f` to all numbers. Takes the file path and the function to apply.

Lines 7–8 — read the numbers

```
1  int count = 0;
2  int *nums = read_numbers(filepath, &count);
```

We pass `&count` (the address of `count`) so that `read_numbers` can write how many numbers it found. After this line, `nums` points to an array of `count` integers.

Lines 9–12 — safety check

```
if (!nums || count == 0) {
    free(nums);
    return 0;
}
```

Two failure cases: the file could not be read (`nums` is `NULL`), or the file was empty (`count == 0`). In either case we free whatever memory we have and return 0. Note that `free(NULL)` is safe in C — it does nothing.

Line 14 — seed the result

```
int result = nums[0];
```

We start with the first number. This is correct because `f` takes two arguments, so we need an initial value to pair with the second number on the first iteration.

Lines 15–17 — the fold

```
for (int i = 1; i < count; i++) {
    result = f(result, nums[i]);
}
```

This is the core of the function. On each step, we call `f` with the accumulated result and the next number, then store the new result. Starting from `i = 1` because `nums[0]` is already in `result`.

For the input `[1, 2, 3, 4, 5]` with `add`:

$$\begin{aligned}
\text{result} &= 1 \\
\text{result} &= \text{add}(1, 2) = 3 \\
\text{result} &= \text{add}(3, 3) = 6 \\
\text{result} &= \text{add}(6, 4) = 10 \\
\text{result} &= \text{add}(10, 5) = 15
\end{aligned}$$

Lines 19–20 — cleanup and return

`free(nums)` releases the heap memory that `read_numbers` allocated. We return the final result.

5 parallel_compute.c

This is the heart of the project. It splits the numbers across `n_proc` child processes, each computes its slice, sends the result back through a pipe, and the parent combines everything.

```
1 #include "compute.h"
2 #include "parse.h"
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6 #include <sys/wait.h>
```

Listing 4: parallel_compute.c — includes and signature

Line 5 — `unistd.h`

Gives us `fork()`, `read()`, `write()`, `close()`, and `pipe()`.

Line 6 — `sys/wait.h`

Gives us `wait()`, which the parent uses to wait for children to finish.

5.1 Step 1 — read the numbers

```
int parallel_compute(const char *filepath, int n_proc, func_t f
    ) {
    int count = 0;
    int *nums = read_numbers(filepath, &count);
    if (!nums || count == 0) {
        free(nums);
        return 0;
    }

    if (n_proc > count) n_proc = count;
```

Same as in `sequential_compute` — we read all numbers first, in the parent process. This is important: we do this *before* forking, so every child inherits a copy of the array automatically when `fork()` duplicates the parent's memory.

The line `if (n_proc > count) n_proc = count` handles the edge case where someone asks for more processes than there are numbers. There is no point creating a process with no numbers to compute, so we clamp `n_proc` down to `count`.

5.2 Step 2 — create the pipes

```
    int (*ctrl_pipe)[2] = malloc(n_proc * sizeof(int[2]));
    int (*res_pipe)[2]  = malloc(n_proc * sizeof(int[2]));
    if (!ctrl_pipe || !res_pipe) {
        free(nums); free(ctrl_pipe); free(res_pipe);
        return 0;
    }

    for (int i = 0; i < n_proc; i++) {
```

```
        if (pipe(ctrl_pipe[i]) < 0 || pipe(res_pipe[i]) < 0) {
            perror("pipe");
            free(nums); free(ctrl_pipe); free(res_pipe);
            return 0;
        }
    }
```

The pipe arrays

```
int (*ctrl_pipe)[2] = malloc(n_proc * sizeof(int[2]));
```

This allocates an array of `n_proc` pairs of integers on the heap. Each pair is one pipe: `ctrl_pipe[i][0]` is the read end and `ctrl_pipe[i][1]` is the write end. We use two separate arrays:

Array	Direction	Purpose
`ctrl_pipe[i]`	parent → child	parent sends `start` and `end` index
`res_pipe[i]`	child → parent	child sends its partial result

Creating the pipes before forking

```
for (int i = 0; i < n_proc; i++) {
    pipe(ctrl_pipe[i]);
    pipe(res_pipe[i]);
}
```

We must call `pipe()` *before* `fork()` for each child. This is critical. When `fork()` is called, the child gets a copy of all open file descriptors — including both ends of both pipes. That is how the parent and child end up sharing the same pipe.

Why this matters

Think of it this way: if you create the pipe *after* forking, only one process knows about it. The pipe has to be created while there is still only one process, so both ends get inherited by the child automatically.

5.3 Step 3 — compute slice sizes

```
    int chunk     = count / n_proc;
    int remainder = count % n_proc;
```

`chunk` is how many numbers each child gets. If the division is not exact, the leftover (`remainder`) will be given to the last child. For example, with 7 numbers and 3 processes:

- `chunk = 7 / 3 = 2`
- `remainder = 7 % 3 = 1`
- Child 0 gets indices `[0, 1]`
- Child 1 gets indices `[2, 3]`
- Child 2 gets indices `[4, 5, 6]` (2 base + 1 remainder)

5.4 Step 4 — fork the children

```
    for (int i = 0; i < n_proc; i++) {
        pid_t pid = fork();
        if (pid < 0) {
            perror("fork");
            free(nums); free(ctrl_pipe); free(res_pipe);
            return 0;
        }

        if (pid == 0) {
            /* ===== CHILD ===== */
            ...
            exit(0);
        }
        /* parent loops again to create the next child */
    }
```

We call `fork()` inside a loop. Each iteration creates one new child. After `fork()` returns:

- If we are in the **child** (`pid == 0`), we do our work and call `exit(0)`.
- If we are in the **parent** (`pid > 0`), the `if` block is skipped and the loop continues to create the next child.

The child calls `exit(0)` at the end, so it never loops further — it terminates. Only the parent keeps looping.

5.5 Step 4a — inside the child

```
            for (int j = 0; j < n_proc; j++) {
                close(ctrl_pipe[j][1]);
                if (j != i) close(ctrl_pipe[j][0]);

                close(res_pipe[j][0]);
                if (j != i) close(res_pipe[j][1]);
            }
```

Why close so many ends?

After `fork()`, the child has a copy of *every* file descriptor — all `2 * n_proc * 2` pipe ends. The child only needs two of them: the read end of its own `ctrl_pipe` and the write end of its own `res_pipe`. Everything else must be closed.

The loop closes them systematically:

- `ctrl_pipe[j][1]` — all write ends of ctrl pipes. The child never writes to any ctrl pipe (that is the parent's job).
- `ctrl_pipe[j][0]` where `j != i` — read ends of ctrl pipes for other children. This child has nothing to do with those pipes.
- `res_pipe[j][0]` — all read ends of result pipes. The child never reads results (that is the parent's job).
- `res_pipe[j][1]` where `j != i` — write ends of result pipes for other children.

Important

This step is the most important thing about pipes. If you leave a write end open somewhere, the process waiting on the read end will block forever. The OS says: "I will not signal end-of-file because someone still has the write end open." Closing every unused end is not optional.

Reading the slice boundaries

```
            int start, end;
            read(ctrl_pipe[i][0], &start, sizeof(int));
            read(ctrl_pipe[i][0], &end,   sizeof(int));
            close(ctrl_pipe[i][0]);
```

The child blocks here until the parent sends the two integers. `read()` receives exactly `sizeof(int)` bytes (4 bytes) into `start`, then another 4 bytes into `end`. After reading both values, we close the read end because we no longer need it.

Computing the partial result

```
            int partial = nums[start];
            for (int k = start + 1; k <= end; k++) {
                partial = f(partial, nums[k]);
            }
```

Exactly the same fold logic as `sequential_compute`, but only over the slice `nums[start..end]`. The child computes its portion completely on its own.

Sending the result back

```
            write(res_pipe[i][1], &partial, sizeof(int));
            close(res_pipe[i][1]);
```

`write()` sends the 4-byte integer through the result pipe to the parent. We then close the write end. Once closed, the parent's matching `read()` will eventually return.

Cleanup and exit

```
            free(nums);
            free(ctrl_pipe);
            free(res_pipe);
            exit(0);
```

The child frees all heap memory it owns, then exits. `exit(0)` terminates the child process cleanly. The `0` means success.

5.6 Step 5 — parent sends slice boundaries

```
    for (int i = 0; i < n_proc; i++) {
        int start = i * chunk;
        int end   = start + chunk - 1;

        if (i == n_proc - 1) end += remainder;
```

```

        write(ctrl_pipe[i][1], &start, sizeof(int));
        write(ctrl_pipe[i][1], &end,   sizeof(int));
        close(ctrl_pipe[i][1]);
        close(ctrl_pipe[i][0]);

        close(res_pipe[i][1]);
    }
```

After the forking loop, only the parent reaches this code (all children have called `exit(0)`).

Computing each child's range

```
int start = i * chunk;
int end   = start + chunk - 1;
if (i == n_proc - 1) end += remainder;
```

For child `i`, the start index is `i * chunk`. The end index is `start + chunk - 1`. For the last child only, we add the remainder so no numbers are lost.

Sending over the pipe

```
write(ctrl_pipe[i][1], &start, sizeof(int));
write(ctrl_pipe[i][1], &end,   sizeof(int));
```

Two separate `write()` calls, each sending one 4-byte integer. On the child's side, two matching `read()` calls receive them in the same order.

Closing the parent's ends

```
close(ctrl_pipe[i][1]);  // done writing to child i
close(ctrl_pipe[i][0]);  // parent never reads from ctrl_pipe
close(res_pipe[i][1]);   // parent never writes to res_pipe
```

The parent closes the write end of `ctrl_pipe` because it is done sending. It closes the read end of `ctrl_pipe` because the parent never reads from there. It closes the write end of `res_pipe` because the parent never writes to there. The only end the parent keeps open is `res_pipe[i][0]` — the read end where it will collect results.

5.7 Step 6 — wait for children, then collect results

```
    for (int i = 0; i < n_proc; i++) {
        wait(NULL);
    }
```

The parent calls `wait(NULL)` once for each child. `wait()` blocks until any one child exits. Calling it `n_proc` times means we wait for all of them.

Why this matters

Why wait before reading? In our code, we actually wait *before* reading the result pipes. This is safe because the children have already written their results and closed their write ends before calling `exit(0)`. So by the time `wait()` returns, all results are

already sitting in the pipes ready to be read.

```
    int *partials = malloc(n_proc * sizeof(int));
    for (int i = 0; i < n_proc; i++) {
        read(res_pipe[i][0], &partials[i], sizeof(int));
        close(res_pipe[i][0]);
    }
```

We allocate a small array for the partial results, then read one integer from each result pipe. After reading, we close the read end.

5.8 Step 7 — aggregate the partial results

```
    int result = partials[0];
    for (int i = 1; i < n_proc; i++) {
        result = f(result, partials[i]);
    }
```

Same left-fold pattern again, but now applied to the partial results instead of the original numbers. The final `result` is the answer.

Note

We use `f` here, not addition. This is important. If the operation is multiply, the partial results must also be combined with multiply. Using addition would give the wrong answer.

5.9 Step 8 — final cleanup

```
    free(partials);
    free(nums);
    free(ctrl_pipe);
    free(res_pipe);
    return result;
```

All four heap allocations are freed: the partials array, the numbers array, and the two pipe arrays. We then return the final answer.

6 test_correctness.c

This file runs 26 test cases that verify both functions are correct before any timing experiments are done.

```c
int add(int a, int b)       { return a + b; }
int multiply(int a, int b) { return a * b; }
int max_f(int a, int b)     { return a > b ? a : b; }
```

Listing 5: test_correctness.c — sample functions

Three sample functions, each matching the `func_t` signature. They are used as arguments to both compute functions during testing.

6.1 write_file — helper to create test input files

```c
static void write_file(const char *path, int *nums, int n) {
    FILE *fp = fopen(path, "w");
    for (int i = 0; i < n; i++) {
        if (i > 0) fprintf(fp, "\n");
        fprintf(fp, "%d", nums[i]);
    }
    fclose(fp);
}
```

Creates a temporary file with one integer per line. The `if (i > 0)` check prevents a newline before the first number. These files are written to `/tmp/` which is a safe location for temporary data on Linux.

6.2 check — test assertion helper

```c
static void check(const char *label, int got, int expected) {
    if (got == expected)
        printf("  PASS  %-45s  got=%d\n", label, got);
    else
        printf("  FAIL  %-45s  got=%d  expected=%d\n", label,
           got, expected);
}
```

Takes a description, the actual result, and the expected result. Prints PASS or FAIL accordingly. The `%-45s` format specifier left-aligns the label in a 45-character field so all the output lines up neatly.

6.3 test_sequential

```c
void test_sequential() {
    int a[] = {1, 2, 3, 4, 5};
    write_file("/tmp/t1.txt", a, 5);

    check("add [1,2,3,4,5]",
          sequential_compute("/tmp/t1.txt", add), 15);
    check("multiply [1,2,3,4,5]",
```

```
            sequential_compute("/tmp/t1.txt", multiply), 120);
    check("max [1,2,3,4,5]",
            sequential_compute("/tmp/t1.txt", max_f), 5);

    int b[] = {42};
    write_file("/tmp/t2.txt", b, 1);
    check("single element 42",
            sequential_compute("/tmp/t2.txt", add), 42);

    int c[] = {10, 7};
    write_file("/tmp/t3.txt", c, 2);
    check("add [10,7]",
            sequential_compute("/tmp/t3.txt", add), 17);
}
```

Tests five scenarios: addition, multiplication, max over five elements, a single element (edge case), and two elements.

6.4 test_parallel

```
void test_parallel() {
    int a[] = {1, 2, 3, 4, 5};
    write_file("/tmp/t1.txt", a, 5);

    check("add [1..5] n_proc=5",
            parallel_compute("/tmp/t1.txt", 5, add), 15);
    check("add [1..5] n_proc=2",
            parallel_compute("/tmp/t1.txt", 2, add), 15);
    check("add [1..5] n_proc=3",
            parallel_compute("/tmp/t1.txt", 3, add), 15);
    check("add [1..5] n_proc=1",
            parallel_compute("/tmp/t1.txt", 1, add), 15);

    int c[] = {1, 2, 3, 4, 5, 6, 7};
    write_file("/tmp/t5.txt", c, 7);
    check("add [1..7] n_proc=3 (N not div by nproc)",
            parallel_compute("/tmp/t5.txt", 3, add), 28);
}
```

The most important test here is the last one — 7 numbers across 3 processes. This tests the remainder logic:

- `chunk = 7 / 3 = 2`, `remainder = 1`
- Child 0: `[1,2]` → partial = 3
- Child 1: `[3,4]` → partial = 7
- Child 2: `[5,6,7]` → partial = 18
- Parent: `f(f(3,7),18) = f(10,18) = 28`

6.5 test_both_agree

```
void test_both_agree() {
```

```c
    int nums[] = {3, 7, 2, 9, 1, 5, 8, 4, 6, 10};
    write_file("/tmp/t6.txt", nums, 10);

    for (int np = 1; np <= 5; np++) {
        int s = sequential_compute("/tmp/t6.txt", add);
        int p = parallel_compute("/tmp/t6.txt", np, add);
        char label[64];
        snprintf(label, sizeof(label),
                 "add [3,7,2,9,1,5,8,4,6,10] n_proc=%d", np);
        check(label, p, s);
    }
}
```

This test does not hardcode an expected value — it just checks that `parallel_compute` returns the same thing as `sequential_compute` for the same input, across five different values of `n_proc`. This is the key correctness guarantee: no matter how many processes you use, the answer must be the same.

6.6 main

```c
int main() {
    test_sequential();
    test_parallel();
    test_both_agree();
    printf("\nDone.\n");
    return 0;
}
```

Runs all three test groups in order. The project requirement is that everything passes before doing any timing experiments.

How to compile and run:

```
gcc -Wall -o test_correctness \
    test_correctness.c sequential_compute.c parallel_compute.c
./test_correctness
```

7 Summary of the full data flow

Here is what happens from start to finish when you call `parallel_compute("file.txt", 3, add)`:

1. Parent reads all numbers from the file into `nums[]`.
2. Parent creates 6 pipes: `ctrl_pipe[0..2]` and `res_pipe[0..2]`.
3. Parent forks Child 0. Child 0 closes all pipes except `ctrl_pipe[0][0]` and `res_pipe[0][1]`, then blocks on `read(ctrl_pipe[0][0], ...)`.
4. Parent forks Child 1. Same but for index 1.
5. Parent forks Child 2. Same but for index 2.
6. Parent sends `(start, end)` through each `ctrl_pipe[i][1]`.
7. Each child wakes up, reads its `start` and `end`, computes its partial result, writes it to `res_pipe[i][1]`, and calls `exit(0)`.
8. Parent calls `wait()` three times.
9. Parent reads three partial results from `res_pipe[0][0]`, `res_pipe[1][0]`, `res_pipe[2][0]`.
10. Parent applies `f` to the three partial results and returns the final answer.

8 Common mistakes this code avoids

Mistake	How we avoided it
`Creating pipes after fork`	All pipes are created in the parent before any fork() call
`Not closing unused pipe ends`	Every child closes all ends it doesn't own in a loop
`Reading from wrong pipe end`	`read(res_pipe[i][0])` — always index [0] for reading
`Using + instead of f to aggregate`	Parent calls `f(result, partials[i])`, not hard-coded addition
`Stack-allocated array returned`	`parse.h` uses `malloc`, not a local array
`Forgetting to wait()`	Parent loops `n_proc` times calling `wait(NULL)`
`Losing malloc pointer on realloc fail`	`parse.h` uses a temp pointer before assigning back